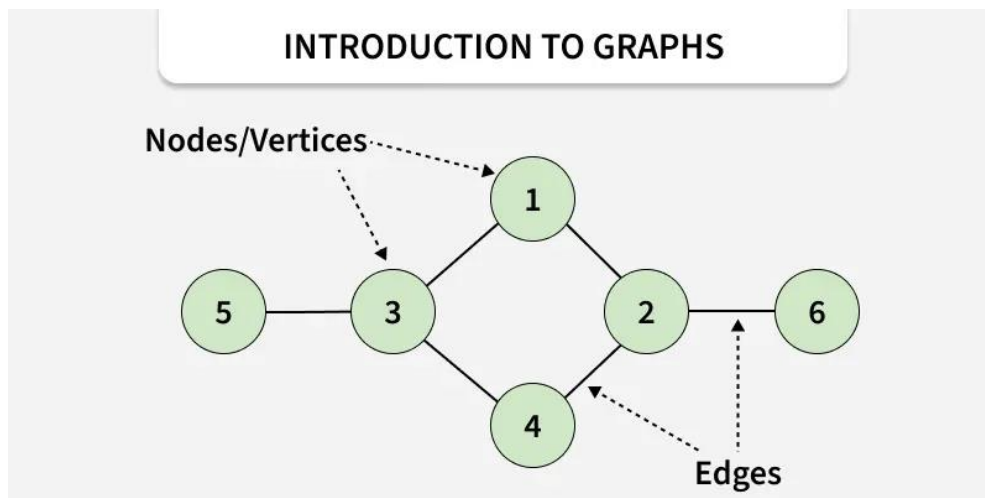


Non-Linear Data Structure

1. Graph

A graph is a non-linear data structure consisting of a finite set of **nodes** (also called **vertices**) and a set of **edges** (links or **arcs**) that connect pairs of vertices. This structure is mainly used to represent data containing a **hierarchical relationship** between **nodes/elements**. The graphs can represent arbitrary, many-to-many relationships.



Graphs are used to represent and solve problems where the data consists of objects and relationships between them, such as:

Social Networks: Each person is a vertex, and relationships (like friendships) are the edges. Algorithms can suggest potential friends.

Maps and Navigation: Locations, like a town or bus stops, are stored as vertices, and roads are stored as edges. Algorithms can find the shortest route between two locations when stored as a Graph.

Internet: Can be represented as a Graph, with web pages as vertices and hyperlinks as edges.

Biology: Graphs can model systems like neural networks or the spread of diseases.

Types Of Graphs in Data Structure and Algorithms

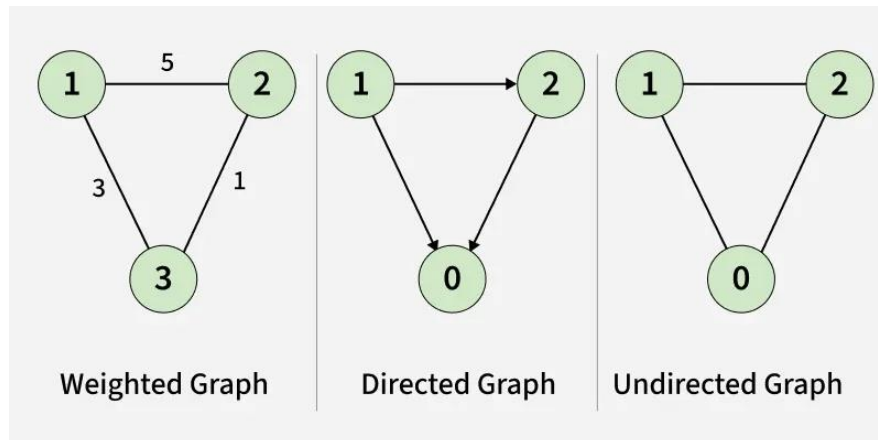
A graph can be divided into multiple categories based on different properties such as edges, direction, connectivity and many others.

Based on weight :

- A weighted graph is a graph where each edge has a number (weight) that represents distance, cost, or time.
- An unweighted graph is a graph where all edges are treated equally, with no extra values like distance or cost.

Based on Edge Direction:

- A Undirected Graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.
- A Directed Graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.



- A directed cyclic Graph is when you can follow a path along the directed edges that goes in circles.
- A loop, also called a self-loop, is an edge that begins and ends on the same vertex. A loop is a cycle that only consists of one edge.

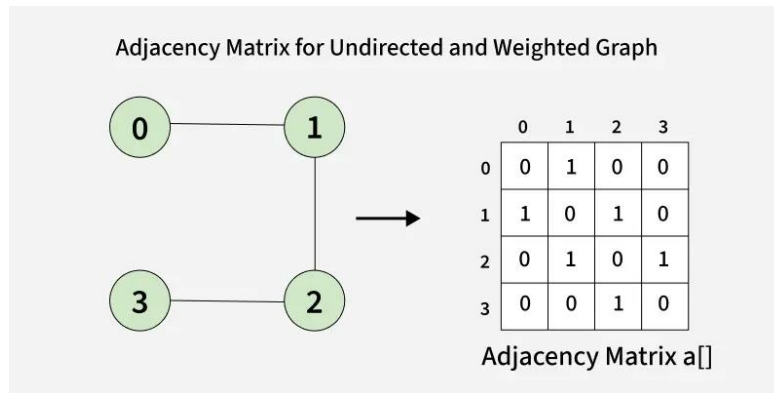
Graph Representations

A Graph representation tells us how a Graph is stored in memory. The more frequently used are:

1. Adjacency Matrix

It's a two-dimensional matrix used with graphs. where each cell on index (i,j) stores information about the edge from vertex i to vertex j. It is more efficient than the adjacency list representation when the graph is dense and we need quick access frequently.

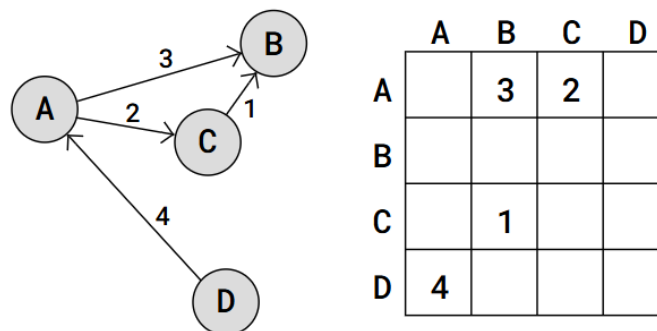
Below is a Graph with the Adjacency Matrix representation next to it.



The adjacency matrix above represents an undirected Graph, so the values '1' only tells us where the edges are. Also, the values in the adjacency matrix is symmetrical because the edges go both ways (undirected Graph).

To create a directed Graph with an adjacency matrix, we must decide which vertices the edges go from and to, by inserting the value at the correct indexes (i,j). To represent a weighted Graph we can put other values than '1' inside the adjacency matrix.

Below is a directed and weighted Graph with the Adjacency Matrix representation next to it.



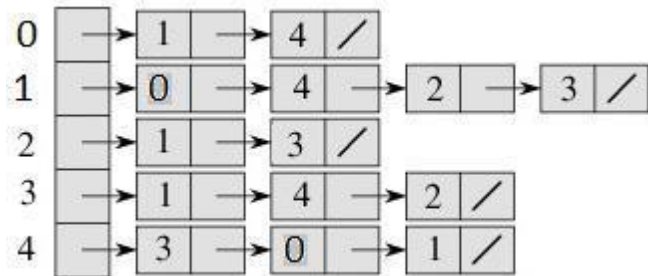
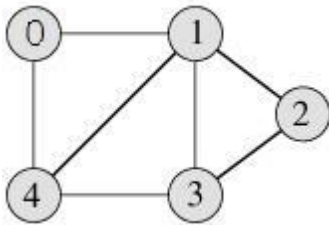
```
vertexData = ['A', 'B', 'C', 'D']
adjacency_matrix = [
    [0, 1, 1, 1], # Edges for A
    [1, 0, 1, 0], # Edges for B
    [1, 1, 0, 0], # Edges for C
    [1, 0, 0, 0]  # Edges for D
]

def print_adjacency_matrix(matrix):
    print("\nAdjacency Matrix:")
    for row in matrix:
        print(row)

print('vertexData:', vertexData)
print_adjacency_matrix(adjacency_matrix)
```

2. Adjacency List:

An array of linked lists is used. Size of the array is equal to number of vertices. Let the array be `array[]`. An entry `array[i]` represents the linked list of vertices adjacent to the *i*th vertex. This representation can also be used to represent a weighted graph. The weights of edges can be stored in nodes of linked lists. Following is adjacency list representation of the above graph.



```
# Adjacency list representation
adjacency_list = { 0: [1, 4], 1: [0, 2, 3, 4], 2: [1, 3], 3: [1, 2, 4], 4: [0, 1, 3] }

def print_adjacency_list (list_graph):
    print("\n Adjacency list graph:")
    for node in list_graph:
        print(node)

print_adjacency_list (adjacency_list)
```

2. Tree

A tree is a nonlinear hierarchical data structure that consists of **nodes** connected by **edges**.

Node: A node is an entity that contains a key or value and pointers to its child nodes.

The last nodes of each path are called leaf nodes or external nodes that do not contain a link/pointer to child nodes.

The node having at least a child node is called an internal node.

Edge: It is the link between any two nodes.

Root: It is the topmost node of a tree.

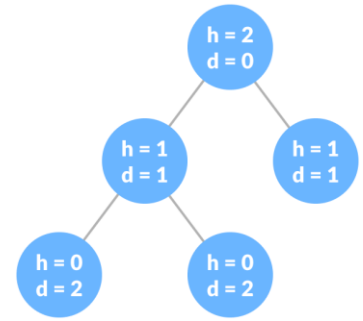
Height of a Node: The height of a node is the number of edges from the node to the deepest leaf (ie. the longest path from the node to a leaf node).

Depth of a Node: The depth of a node is the number of edges from the root to the node.

Height of a Tree: The height of a Tree is the height of the root node or the depth of the deepest node.

Degree of a Node: The degree of a node is the total number of branches of that node.

Forest: A collection of disjoint trees is called a forest.

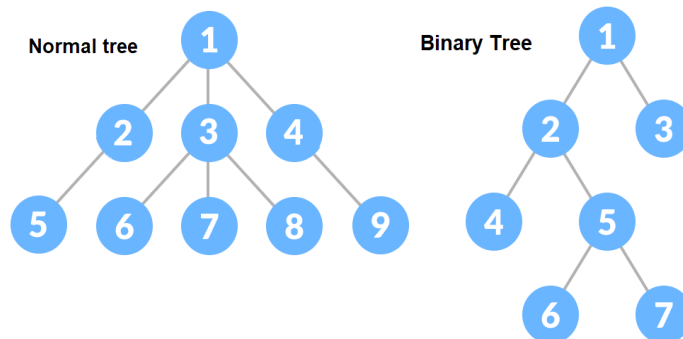


Types of Tree

- In a **normal tree**, every node can have any number of children.
- A **binary tree** is a special type of tree data structure in which every node can have a maximum of 2 children. One is known as a left child and the other is known as right child.

Binary Tree

A binary tree is a tree data structure in which each parent node (element) can have at most two children.



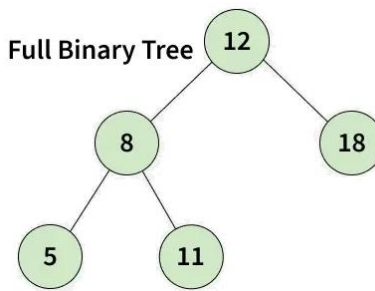
Types of Binary Trees

1. Based on the number of children nodes:

- Full Binary Tree

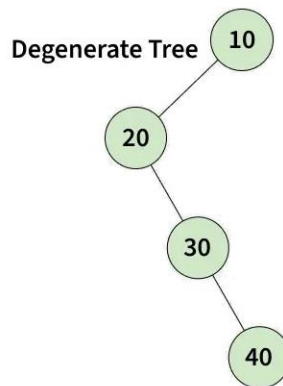
A Binary Tree is a full binary tree if every node has 0 or 2 children. We can also say a full binary tree is a binary tree in which all nodes except leaf nodes have two children.

A full Binary tree is a special type of binary tree in which every parent node/internal node has either two or no children. It is also known as a proper binary tree.



- **Degenerate (or pathological) Tree**

A Tree where every internal node has one child. Such trees are performance-wise same as linked list. A degenerate or pathological tree is **a tree having a single child** either left or right.



- **Skewed Binary Tree**

A skewed binary tree is a pathological / degenerate tree in which the tree is either dominated by the left nodes or the right nodes. Thus, there are two types of skewed binary tree: left-skewed binary tree and right-skewed binary tree.

2. On the basis of level completion:

- **Complete Binary Tree**

A Binary Tree is a Complete Binary Tree if all the levels are completely filled except possibly the last level and the last level is filled from left side.

A complete binary tree is just like a full binary tree, but with two major differences:

- Every level except the last level must be completely filled.
- All the leaf elements must lean towards the left.

- **Perfect Binary Tree**

A Binary tree is a Perfect Binary Tree in which all the internal nodes have two children and all leaf nodes are at the same level.

A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level.

In a Perfect Binary Tree, the number of leaf nodes is the number of internal nodes plus 1.

- **Balanced Binary Tree**

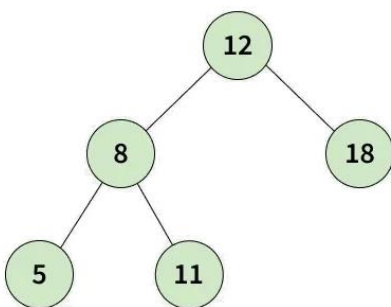
A binary tree is balanced if the height of the tree is $O(\log n)$ where n is the number of nodes.

For example, the AVL tree maintains $O(\log n)$ height by making sure that the difference between the heights of the left and right subtrees of every node is at most 1.

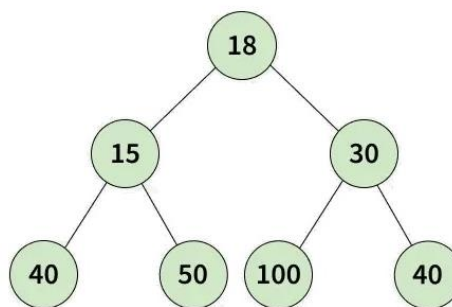
Following is an example of balanced binary tree. Here,

$$d = \text{depth of node} = |\text{Height of Left child} - \text{Height of Right child}|.$$

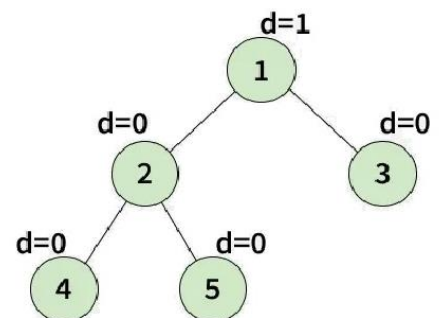
Complete Binary Tree



Perfect Binary Tree



Balanced Binary Tree



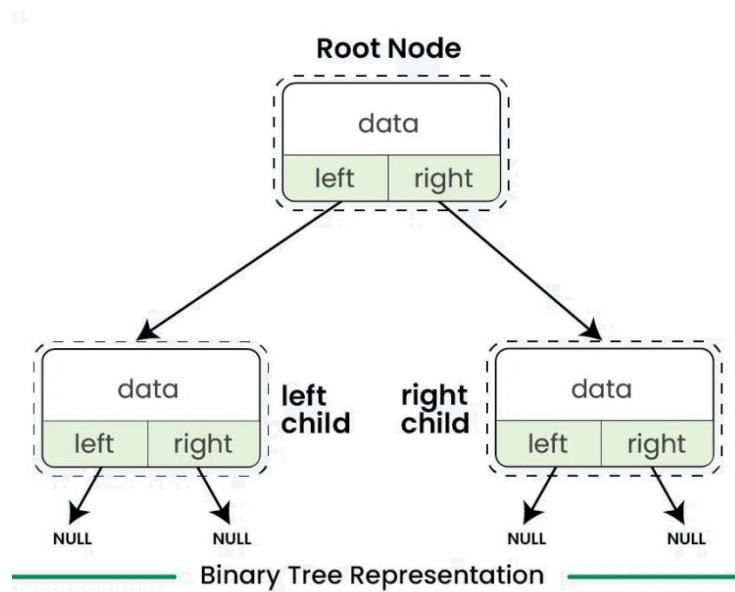
Representation of Binary Trees

Binary trees can be represented in multiple ways, each with its own advantages, depending on the use case. Let's explore the two common methods: linked node representation and array implementation.

1. Linked Node Representation

This is the simplest way to represent a binary tree. Each node contains **data** and pointers to its **left** and **right** children.

This representation is mostly used to represent binary tree with multiple advantages. The most common advantages are given below.



Advantages:

- It can easily grow or shrink as needed, so it uses only the memory it needs.
- Adding or removing nodes is straightforward and requires only pointer adjustments.
- Only uses memory for the nodes that exist, making it efficient for sparse trees.

Disadvantages:

- Needs extra memory for pointers.
- Finding a node can take longer because you have to start from the root and follow pointers.

Create/Declare a Node of a Binary Tree:

Syntax to declare a node of "linked list representation of binary tree" in different languages:

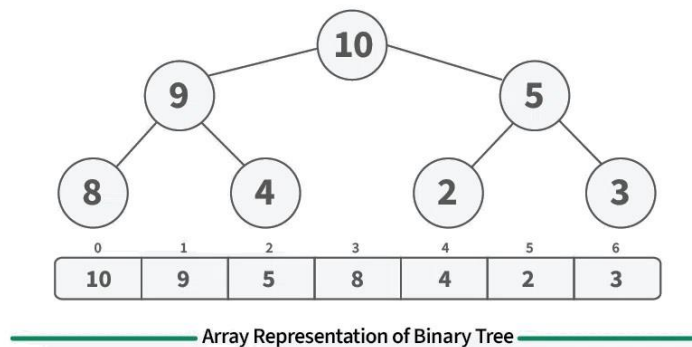
Python

```
# Node structure of each node of the binary tree.  
class Node:  
    def __init__(self, val):  
        self.data = val  
        self.left = None  
        self.right = None
```


2. Array Representation

Array Representation is another way to represent binary trees, especially useful when the tree is complete (all levels are fully filled except possibly the last, which is filled from left to right). In this method the elements are represented using arrays.:

- The Root Node: Stored at index 0
- For any element in position i ,
 - The left child is in position $2i$,
 - The right child is in position $(2i + 1)$,
 - The parent is in position $(i/2)$.



Advantages:

- Easy to navigate parent and child nodes using index calculations, which is fast
- Easier to implement, especially for complete binary trees.

Disadvantages:

- You have to set a size in advance, which can lead to wasted space.
- If the tree is not complete binary tree then many slots in the array might be empty, this will result in wasting memory
- Not as flexible as linked representations for dynamic trees.

Comparison of Linked list vs Array Representation of Binary Tree

Feature	Linked Node Representation	Array Representation
Memory Usage	Uses extra memory for pointers	Efficient for complete trees
Ease of Implementation	Simple and intuitive	Simple but limited to complete trees
Dynamic Size	Easily grows and shrinks	Not flexible for dynamic sizes
Access Time	Slower access for specific nodes	Fast access using indices
Suitable For	Any binary tree structure	Complete binary tree